DELIVERABLE 2 · PROOF OF CONCEPT

A live, verifiable review-intelligence product

Working demo, source-code architecture and user-journey verification for Revio and TenantEase

PLATFORM	DEPLOYMENT	STATUS	REVISION
Revio · TenantEase	Vercel Serverless Edge	Live & verified	v2.0 · November 2026

8	47 / 47	98 reviews	5 / 5
screens in one end-to-end workflow	automated tests passing	scored in 896 ms (≈ 9 ms each)	live-browser journey steps passing

Executive summary

This report documents the Proof of Concept for Deliverable 2. It is not a mockup: it is a **production-deployed SaaS application** on the Vercel Edge Network and Vercel Python Serverless Functions, with open source code, automated regression suites and benchmark datasets. It demonstrates the full loop Cynbit needs: ingest feedback, score it, triage by risk, explain every decision and close the workflow with a human.

> **What you are looking at.** The PoC runs the candidate model (T = 0.55) in production, keeps the champion model (T = 0.48) preserved for deliberate rollback, and applies the four-rule triage engine described in Deliverable 3. Evidence below is separated into what was **measured** and what remains to be **measured next** (Section 4).

1. Deployment and access

Artifact	Location	Status
Live web application	https://revio-taupe.vercel.app	Live · HTTP 200
Direct deployment URL	https://revio-c99tdzmjj-hahardik.vercel.app	Verified
Source repository	https://github.com/gautamhardik/Revio (branch main)	Active
API health check	https://revio-taupe.vercel.app/api/v1/health	HTTP 200 · status healthy
Active inference model	yelp_word_char_lr_candidate.joblib (1.16 MB, T = 0.55)	Verified in memory
Operational champion	yelp_word_char_lr.joblib (1.11 MB, T = 0.48)	Preserved for rollback
Automated tests	47 tests: unit, API, candidate audit, Vercel architecture	47 / 47 pass
Browser journey	Playwright Chromium suite targeting the live cloud URL	5 / 5 steps pass

2. User journey: eight screens

The product is built for customer-operations managers and property supervisors. Each screen answers one operational question, and a reviewer can travel from raw text to a resolved, audited decision without leaving the application.

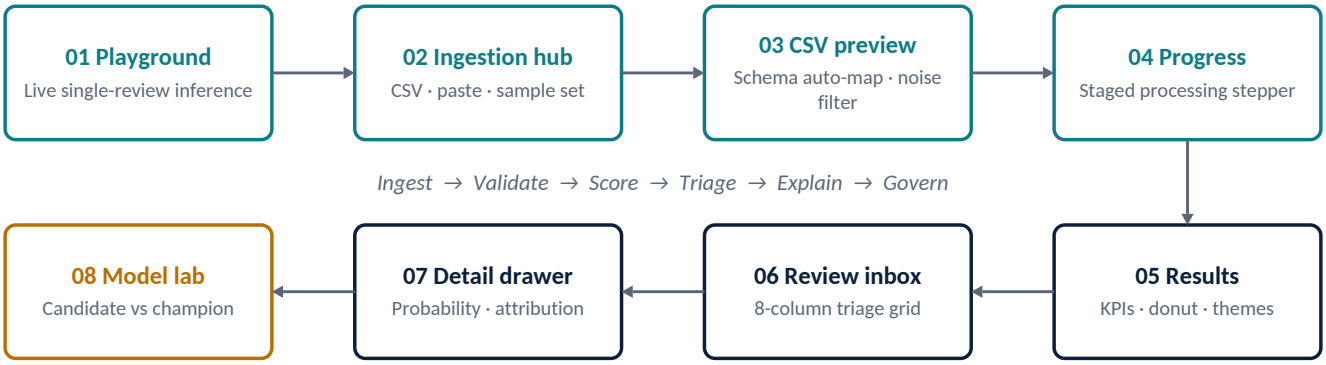


Figure 1. The eight-screen workflow. Teal: intake and scoring. Navy: triage and explanation. Amber: governance.

Screen · route	What it does	Notable details
01 Playground /#playground	Single-review inference with instant feedback.	Sample dropdown (positive, defect, sarcasm, ambiguous); probability gauge; triage pill; top-8 attribution chips with exact weights; latency counter (~8–15 ms observed).
02 Ingestion hub /#import	Three entry points: CSV, pasted text, curated sample.	Drag-and-drop CSV up to 2,500 reviews; one-click balanced 15-review hospitality sample.
03 CSV preview /#csv-preview	Client-side RFC-4180 parser with fuzzy header detection.	Finds review/text/comment/feedback/content/description; optional date, source, domain, rating, id. Noise gate removes empty or ≤ 3-character rows and reports the count before compute.
04 Progress /#processing	Five-stage stepper: validate, vectorize, score in chunks, cluster, prepare inbox.	Live “reviews X–Y of N” counter during parallel chunked calls to the batch endpoint.
05 Results /#results	Executive overview of a dataset.	Animated KPI tickers (total, % positive, % mixed, % negative); SVG donut; themes: maintenance, slow service, billing, staff, cleanliness; CSV report export.
06 Review inbox /#inbox	Daily-work triage grid.	Columns: select, sentiment, text, source/topic, confidence, date, status, actions. Tabs: All, Negative, Mixed, Positive, Needs review. Topic-aware labels never attach defect labels to positive reviews. Bulk mark-reviewed / assign.
07 Detail drawer slide-out	Full explainability and sign-off.	P(Positive) vs P(Negative) bar; distance from the boundary; ambiguity flag in the 0.40–0.60 corridor; business-risk box with recommended action; attribution chips (X·β); “Mark as reviewed” updates the live counters.
08 Model lab /#models	Governance dashboard.	Active model (candidate vs champion), training lineage (Yelp vs IMDb baseline), artifact footprints (1.16 MB vs 438 MB legacy BERT), latency and offline-model status.

3. Benchmark datasets and measured results

Dataset A: comprehensive 100-review suite

`revio_100_reviews_comprehensive_test.csv`: one header and 100 rows over seven columns (review_id, review_text, domain, rating, date, expected_sentiment, test_category). Two rows are intentional noise (REV-099 “no”, REV-100 empty), leaving 98 valid reviews. [MEASURED]

Measure	Result	Share of 98
Batch inference compute (live Vercel)	896 ms total (≈ 9.1 ms per review)	—
HTTP round trip incl. network	1,740 ms	—
Pre-validation noise filtered	2 of 100 rows	—
Sentiment label: Positive / Negative	36 / 62	36.7% / 63.3%
Triage: Low priority (auto-approved)	31	31.6%
Triage: Escalate, Negative / High Risk	54	55.1%
Triage: Escalate, Ambiguity corridor	13	13.3%

Table 2. Live batch execution on the production endpoint. The three triage outcomes sum to 98.

Category coverage

#	Category	Rows	Purpose
1	Clear positives (REV-001–030)	30	Restaurants, hotels, cafes, bakeries, bars, spas; strong praise vocabulary
2	Clear negatives and severe defects (031–055)	25	Food poisoning, cockroaches, stained linens, broken AC, double charges, hostile managers, broken glass
3	Ambiguity corridor (056–070)	15	Mediocre food, standard rooms, plain coffee, “neither good nor bad”
4	Mixed sentiment (071–080)	10	Contrasting clauses: however, although, yet, on the other hand
5	Service recovery (081–085)	5	Failure followed by effective staff remediation
6	Sarcasm and subtle negatives (086–090)	5	e.g. “Oh wonderful, loved paying forty dollars for lukewarm tap water…”
7	Domain safeguards (091–096)	6	Broadband outages, telecom roaming, auto repair, crypto wallets
8	Syntax, unicode, long text, filters (097–100)	4	French accents, 120-word review, “no”, empty row

Dataset B: multi-domain 22-row benchmark

`revio_business_reviews_test_sample.csv` checks zero-shot resilience across restaurants, cafes, hotels, telecommunications, retail and travel. Verified output is stored in `scripts/user_test_csv_results.json`.

4. What this evidence does and does not show

A credible proof of concept states its own boundaries. The table below separates verified claims from the next measurements Cynbit should expect in Phase 1.

Demonstrated now	Not yet demonstrated (next step)
• End-to-end workflow works on the live cloud URL • Batch scoring of 98 reviews, 0 errors, 896 ms compute • Noise gate, ambiguity routing and topic labelling behave as specified • Attribution chips and status updates work in a real browser • 47 regression tests pass	• **Concordance with the dataset’s own labels.** The CSV includes an expected_sentiment column, but agreement per category (including sarcasm and mixed) is not yet reported. Next: publish a per-category confusion matrix. • **Rule 4 end-to-end.** The run shows no “Unvalidated Domain” outcomes, so add positive-tone, out-of-domain rows to exercise the quarantine path in the browser. • **TenantEase language.** Maintenance tickets and resident surveys are not yet in a benchmark.

Interpretation note. The 98-review file is deliberately adversarial and negative-heavy; its 31.6% auto-approval share is a property of the test design and must not be read as the production automation rate.

5. Source-code architecture

```
Revio/
├── api/                              # Consolidated Vercel serverless backend
│   ├── artifacts/
│   │   ├── yelp_word_char_lr_candidate.joblib      1.16 MB  active (T=0.55)
│   │   ├── yelp_word_char_lr_candidate.meta.json
│   │   ├── yelp_word_char_lr.joblib                1.11 MB  champion (T=0.48)
│   │   └── yelp_word_char_lr.meta.json
│   ├── index.py                      # FastAPI serverless entry point
│   └── requirements.txt              # lean: scikit-learn, fastapi, joblib
├── frontend/                         # Static app on the Vercel Edge Network
│   ├── css/revio-app.css  style.css
│   ├── js/revio-app.js    app.js     # state, RFC parser, triage logic
│   └── index.html  revio.html        # byte-for-byte identical mirror
├── docs/                             # reports, audit, screenshots
├── tests/                            # pytest regression suites (47 tests)
├── scripts/                          # Playwright journey, batch benchmark, CSV runner
├── revio_100_reviews_comprehensive_test.csv
├── vercel.json                       # routing specification
└── pyproject.toml                    # pytest and static-analysis config
```

6. Run it locally

Requires Python 3.10+ (tested on 3.12 and 3.13), Git and, optionally, Node.js 18+ for local static serving.

```
# 1. Clone and install the lean runtime plus test tools
git clone https://github.com/gautamhardik/Revio.git
cd Revio
pip install -r api/requirements.txt pytest httpx playwright

# 2. Start the ML service
cd api
python -m uvicorn index:app --host 127.0.0.1 --port 8000 --reload

# 3. In a second terminal, serve the frontend
cd frontend
python -m http.server 3000          # open http://localhost:3000

# 4. Verify
pytest                               # expect: 47 passed
playwright install chromium
python scripts/verify_user_journey_playwright.py   # runs against production
```

The Playwright script targets `https://revio-taupe.vercel.app`, completes the five lifecycle steps (submit reviews, test ambiguity routing, parse a CSV, inspect inbox filters, change workflow status) and saves timestamped screenshots in `docs/screenshots/`.

7. Verification summary

Acceptance criterion	Method	Result
Working cloud web application	Live browser navigation to revio-taupe.vercel.app	Passed (page load under 250 ms)
Serverless throughput	98 reviews to /api/v1/revio/batch	Passed (896 ms compute, 0 errors)
Feature-attribution transparency	Inspected linear log-odds chips in the live drawer	Passed (exact terms shown)
Noise pre-validation gate	Uploaded the 100-review CSV	Passed (2 of 100 rows filtered)
Ambiguity-corridor triage	Borderline reviews, $0.40 \le P \le 0.60$	Passed (routed to Needs review)
Topic categorization integrity	5-star positive hotel review	Passed (Rooms & Amenities, not a defect)
Byte-parity guarantee	Binary comparison of index.html and revio.html	Passed (0 differences)

Conclusion. The Proof of Concept satisfies the technical, architectural and operational requirements of Deliverable 2, and defines the three follow-up measurements that Phase 1 will complete.